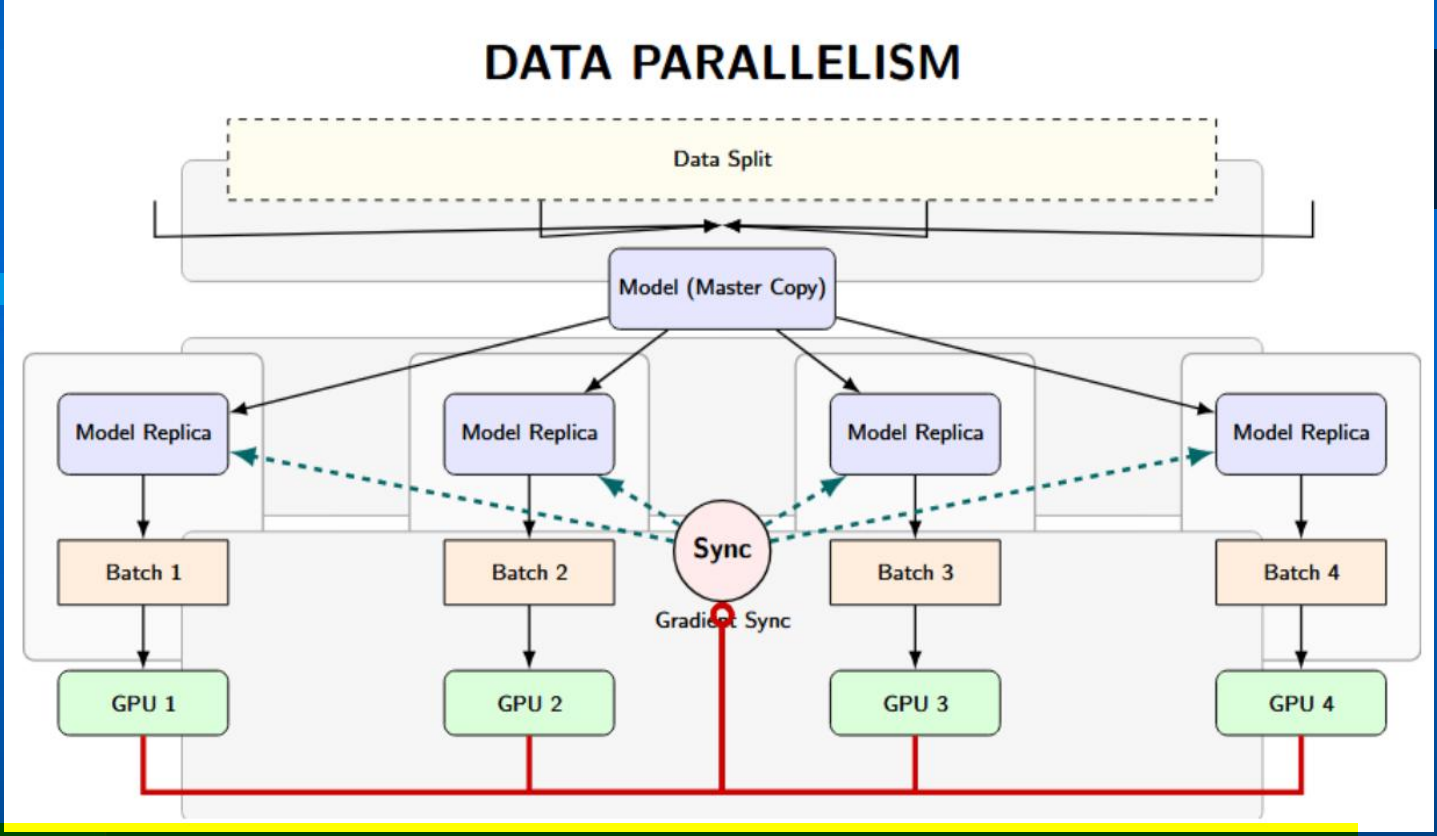
Data Parallelism

Data Parallelism

- Data parallelism is a parallel computing technique where the data or computational workload is divided into chunks that are distributed to multiple processing units.
- Each device performs the same operation on different subsets of the data simultaneously.
- Data parallelism boosts training and processing speeds with more devices.



- **Downward arrows:** Set up and distribute the work and the models.
- **Upward/sync arrows:** Aggregate results so all model copies learn together.

How Data Parallelism Works

Replicate the Model: First, we load an identical copy of the model on each device. By doing this, each device processes its own data simultaneously during the forward and backward passes.

Split the Data: Training dataset is split into N equal-sized chunks, where N is the number of parallel workers (GPUs). Each worker (GPU) receives a different slice of the data.

Parallel Processing: Each GPU processes its data subset in parallel. All the GPUs compute the forward pass (predictions) and backward pass (gradients) simultaneously, without communication during this step. The model replicas get different data.

Gradient Synchronization: After completing local computation for a training step, the workers all synchronize and consistently update the model.

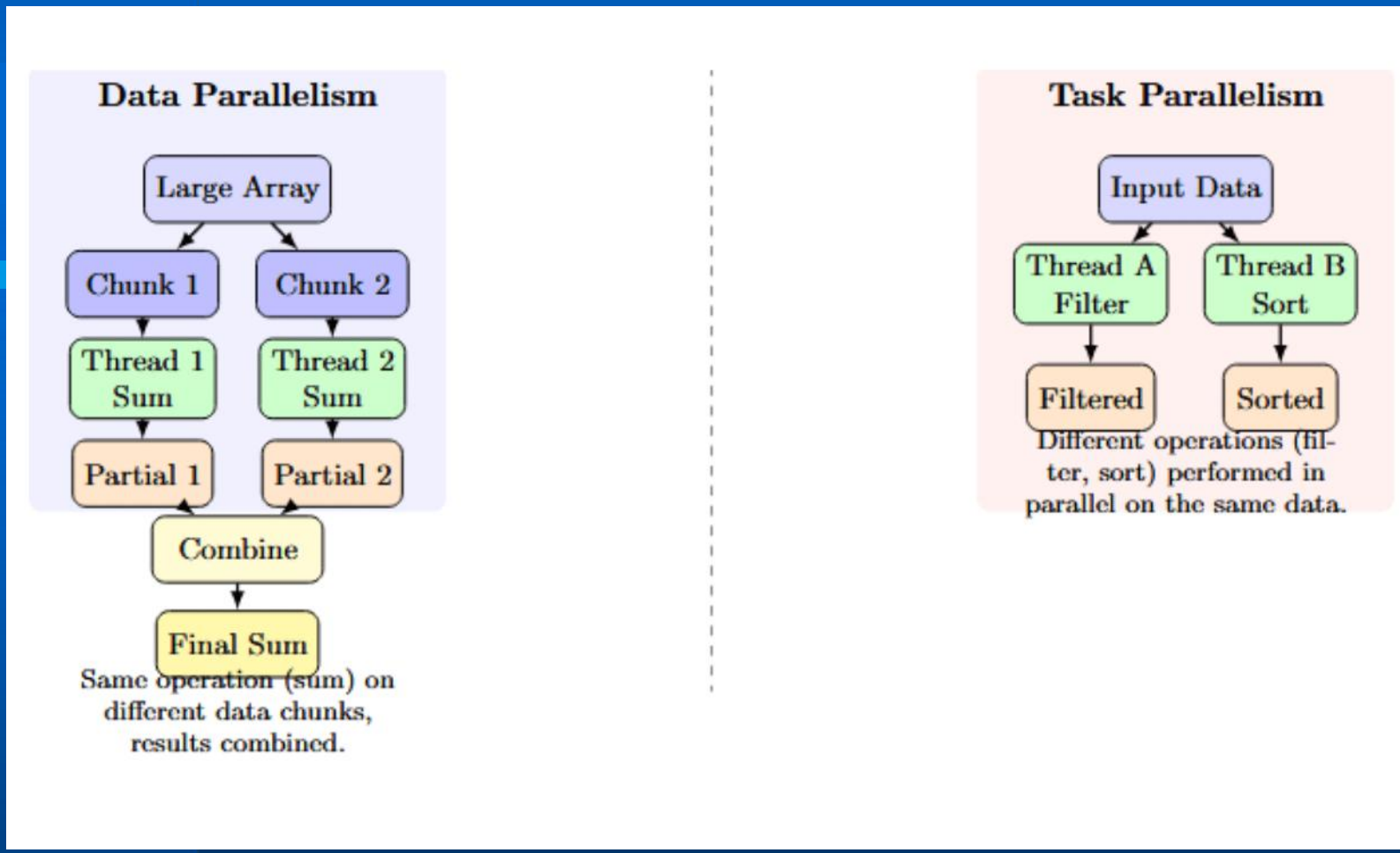
Model Update: Each GPU (or master process) uses the aggregated gradients to update the model weights.

Repeat: Load the next batch of data and repeat the process (split data, parallel forward/backward, sync, update). Continue until training is complete.

Data vs Task Parallelism

Data Parallelism: All workers are doing the same task or operation, but on different subsets of the data simultaneously.

Task Parallelism: Multiple workers (threads or processes) perform distinct tasks, on the same or different subsets of data, in parallel.



Data Parallelism: Key Advantages and Limitations

Pros	Cons
Speed-Up and Efficiency Tasks complete faster by dividing work over multiple processors.	**Memory Overhead** Each worker holds a full copy of the model or data, causing duplicated memory usage and limiting scalability for very large models.
Simplicity of Replication The same code or model runs on each worker; frameworks handle splitting and merging, so don't need major code changes.	**Communication Overhead** Combining results (like synchronizing gradients) adds network traffic, which can become a bottleneck as model or cluster size grows.
Scalability Easy to handle more data or bigger models by adding more workers; widely used for scaling to hundreds or thousands of machines.	**Synchronization Penalty** In synchronous setups, the fastest workers must wait for the slowest, making performance sensitive to hardware imbalance or stragglers.
Fault Isolation If one worker fails, only its portion of the data is lost. With checkpointing, work can resume with minimal loss, improving reliability.	**Diminishing Returns** Adding more workers eventually yields less benefit due to communication and merge overhead, and can affect ML model quality if batches become too large.
Balanced Workload Splitting by data often naturally balances the, keeping all processors equally busy.	**Applicability** Not all problems split easily by data; tasks with strong dependencies or that require global state need different parallelization strategies.

Advanced Processor Technology

Major Processor Families

- **CISC Processor**
- **RISC Processor**

What is RISC?

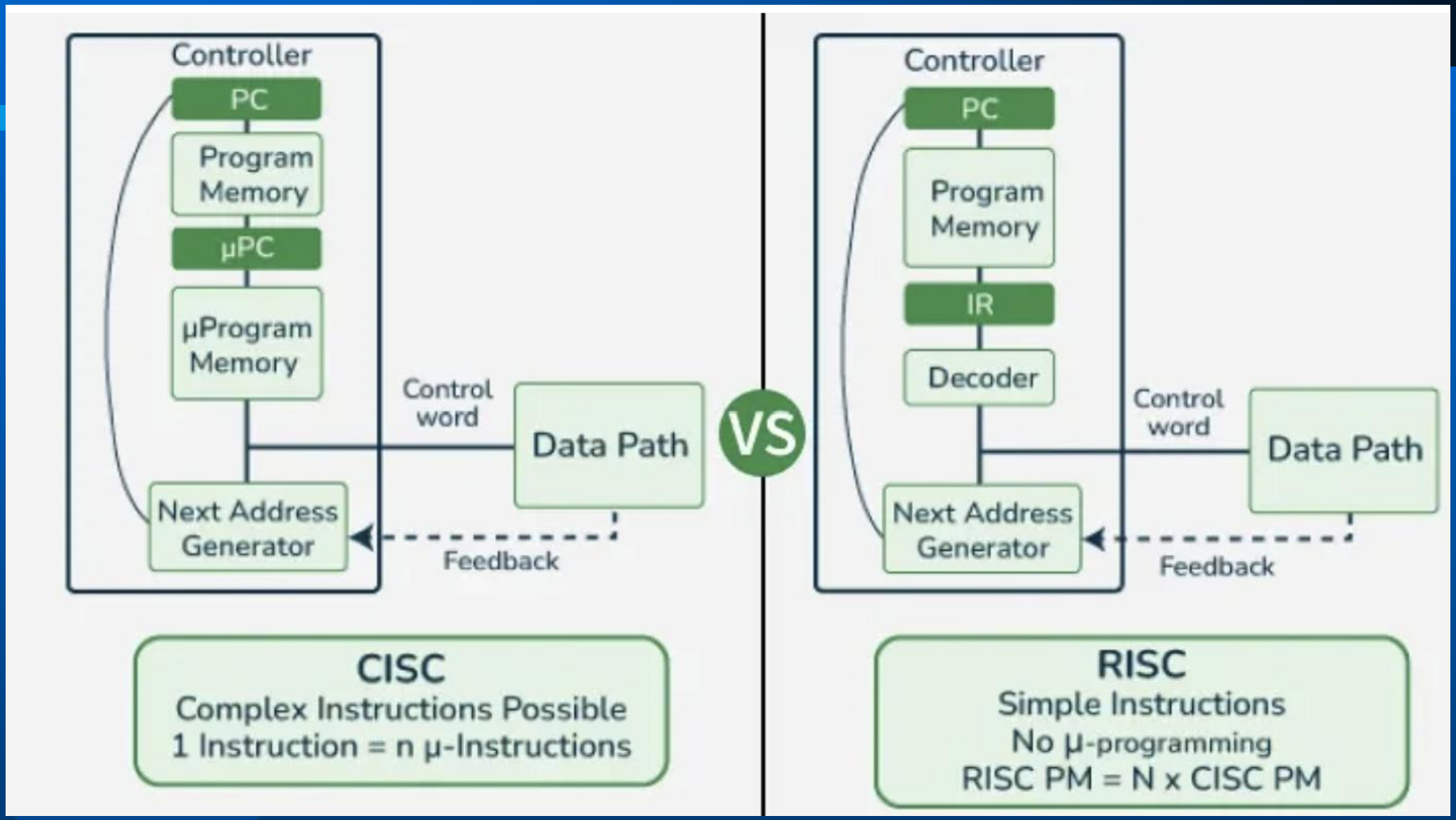
RISC (Reduced Instruction Set Computer)

- **Simple instructions**: Uses a small set of basic instructions that run fast.
- **One clock cycle**: Most instructions finish in a single clock cycle.
- **More registers**: Has a large number of registers to hold data.
- **Examples**: ARM, MIPS, and SPARC

What is CISC?

CISC (Complex Instruction Set Computer)

- **Complex instructions**: Uses large, multi-step instructions that can do many tasks at once.
- **Multiple clock cycles**: Takes more than one clock cycle to finish an instruction.
- **Direct memory access**: Can read and write data directly in the main memory.
- **Examples**: Intel x86 and AMD processors.


Controller
PC
Program Memory
μPC
μProgram Memory
Next Address Generator
Control word
Data Path
Feedback
VS
Controller
PC
Program Memory
IR
Decoder
Next Address Generator
Control word
Data Path
Feedback
CISC
Complex Instructions Possible
1 Instruction = n μ-Instructions
RISC
Simple Instructions
No μ-programming
RISC PM = N x CISC PM

CISC: "Give the hardware complex instructions so the programmer doesn't have to write as much code."
RISC: "Keep the hardware simple and fast, and let the software compiler figure out how to break down complex tasks into simple steps."

CISC	RISC
Emphasis on hardware	Emphasis on software
Multiple instruction sizes and formats	Instructions of same set with few formats
Less registers	Uses more registers
More addressing modes	Fewer addressing modes
Extensive use of microprogramming	Complexity in compiler
Instructions take a varying amount of cycle time	Instructions take one cycle time
Pipelining is difficult	Pipelining is easy

Advanced Processor Technology

- **Super-Scalar Processor**
  - **A special subclass of RISC Processor**
  - **Allow multiple instructions to be issued simultaneously during each cycle**

A super-scalar processor has multiple execution units (like multiple ALUs for math, or load/store units) built into it.

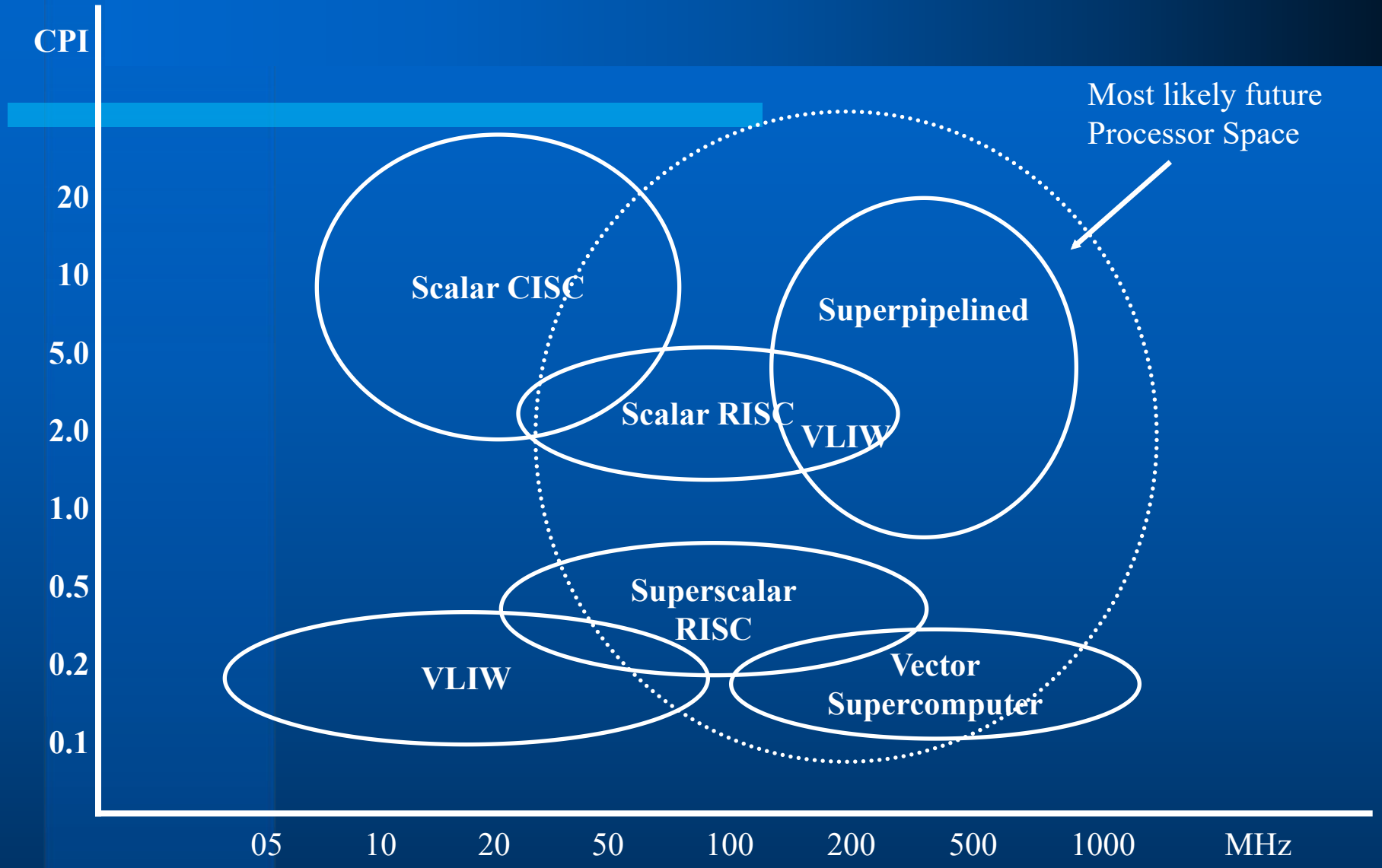
Advanced Processor Technology

- **VLIW (Very Long Instruction Word) Processor**
  - Has more functional unit than that of a superscalar processor
- **Super-Pipelined Processor**
  - Uses multiphase clock
- **Vector Processor**
  - Uses multiple functional units for concurrent scalar and vector operations
- **Symbolic Processor**

Advanced Processor Technology

- **Design Space of Processors**
  - Design space
  - Instruction Pipelines
  - Processors and Coprocessors
- **Instruction-Set Architectures**
  - Complex Instruction Set
  - Reduced Instruction Set
  - Architectural Distinction

The Design Space



Instruction Pipeline

Some definitions

- **Instruction pipeline cycle**
  - **Clock period of the instruction pipeline**
- **Instruction issue latency**
- **Instruction issue rate**
- **Simple operation latency**
- **Resource conflict**

Instruction Pipeline Cycle

In a pipelined processor, several instructions are processed at the same time, but each is in a different stage of execution.

Consider a **5-stage pipeline**:

- IF – Instruction Fetch
- ID – Instruction Decode
- EX – Execute
- MEM – Memory Access
- WB – Write Back

I1, I2, I3, I4

Clock Cycle	IF	ID	EX	MEM	WB
Cycle 1	I1				
Cycle 2	I2	I1			
Cycle 3	I3	I2	I1		
Cycle 4	I4	I3	I2	I1	
Cycle 5		I4	I3	I2	I1
Cycle 6			I4	I3	I2
Cycle 7				I4	I3
Cycle 8					I4

Each **row** represents **one instruction pipeline cycle**.
During every cycle, every stage works simultaneously.

Clock Period of the Instruction Pipeline

The **Clock Period** is the **time required for one pipeline cycle**.

$$\text{Clock Period} = \frac{1}{\text{Clock Frequency}}$$

Clock frequency: 4 GHz

Clock period = 1/4 GHz = 0.25 ns

Instruction Issue Latency

Instruction Issue Latency is the **number of clock cycles between issuing two consecutive instructions from the instruction issue unit.**

Cycle 1 → I1

Cycle 2 → -

Cycle 3 → I2

Cycle 4 → -

Cycle 5 → I3

Issue latency: 2 Clock Cycles

Instruction Issue Rate

Instruction Issue Rate is the **number of instructions that a processor can issue in one clock cycle.**

$$\text{Issue Rate} = \frac{\text{Instructions Issued}}{\text{Clock Cycle}}$$

Cycle 1: I1, I2, I3, I4

Issue Rate: 4 Instructions/Cycle

Modern CPUs

Modern superscalar processors may issue

- 2 instructions/cycle
- 4 instructions/cycle
- 6 instructions/cycle
- 8 instructions/cycle

Simple Operation Latency

It is the number of clock cycles required for a simple operation (such as integer addition or subtraction) to produce its result after it begins execution.

Integer Addition: ADD R1,R2,R3

add r2 and r3 and keep result in r1

Suppose addition needs: 2 Clock Cycles

Cycle 1: Execute begins

↓

Cycle 2: Calculation

↓

Cycle 3: Result available

Latency: 2 Cycles

Resource Conflict

A **Resource Conflict** (also called a **Structural Hazard**) occurs when **two or more instructions require the same hardware resource at the same time, but the processor has only one instance of that resource.**

Only one multiplier exists.
Instruction 1: Multiply
Instruction 2: Multiply

Both arrive together.

One Multiplier
↓
Conflict
↓
Second instruction stalls

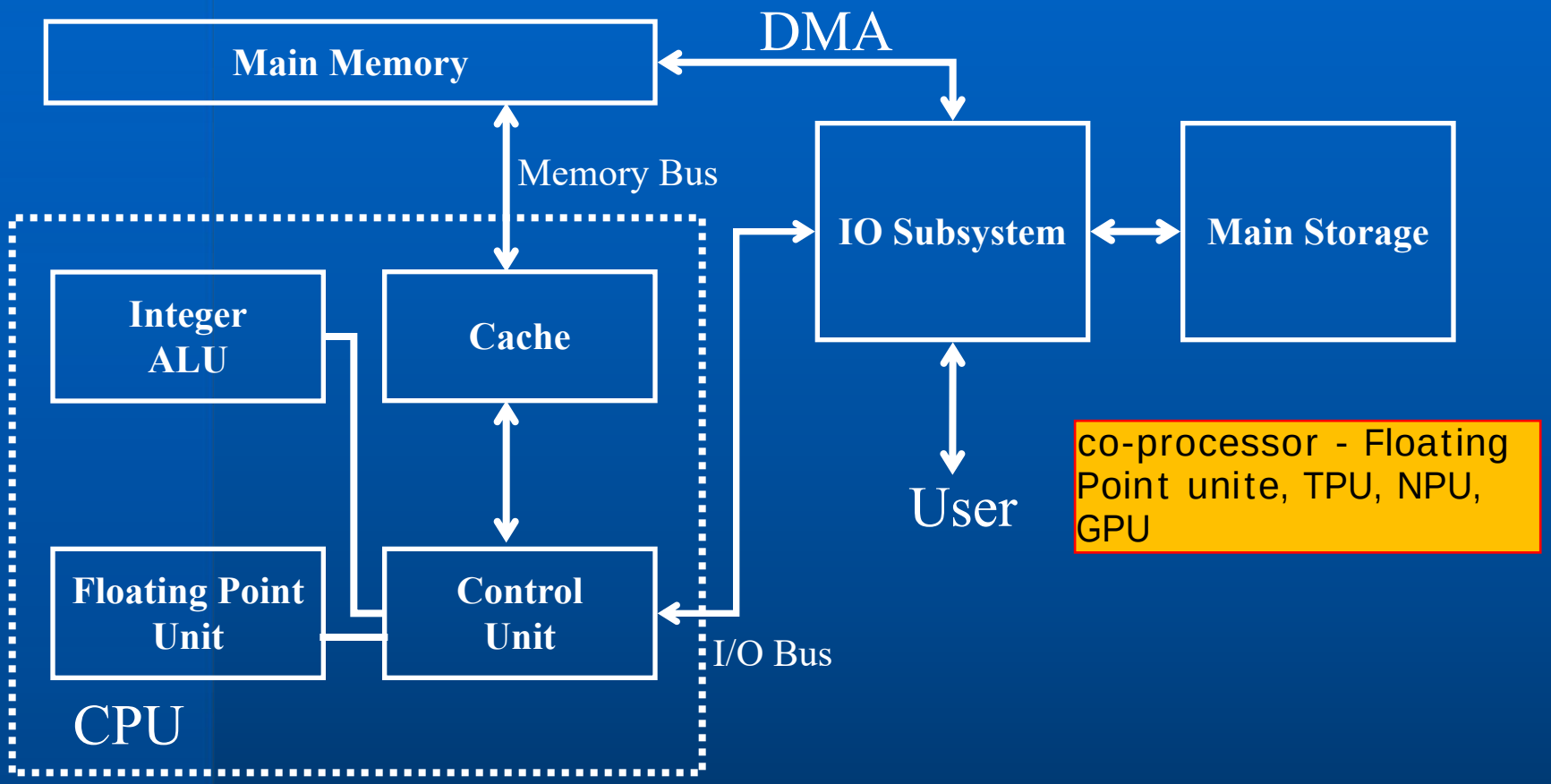
Resource conflicts can be reduced by:

- Adding multiple execution units (e.g., multiple ALUs or multipliers)
- Separating instruction and data memories
- Using resource scheduling
- Employing out-of-order execution to issue other ready instructions while waiting

Summary Table

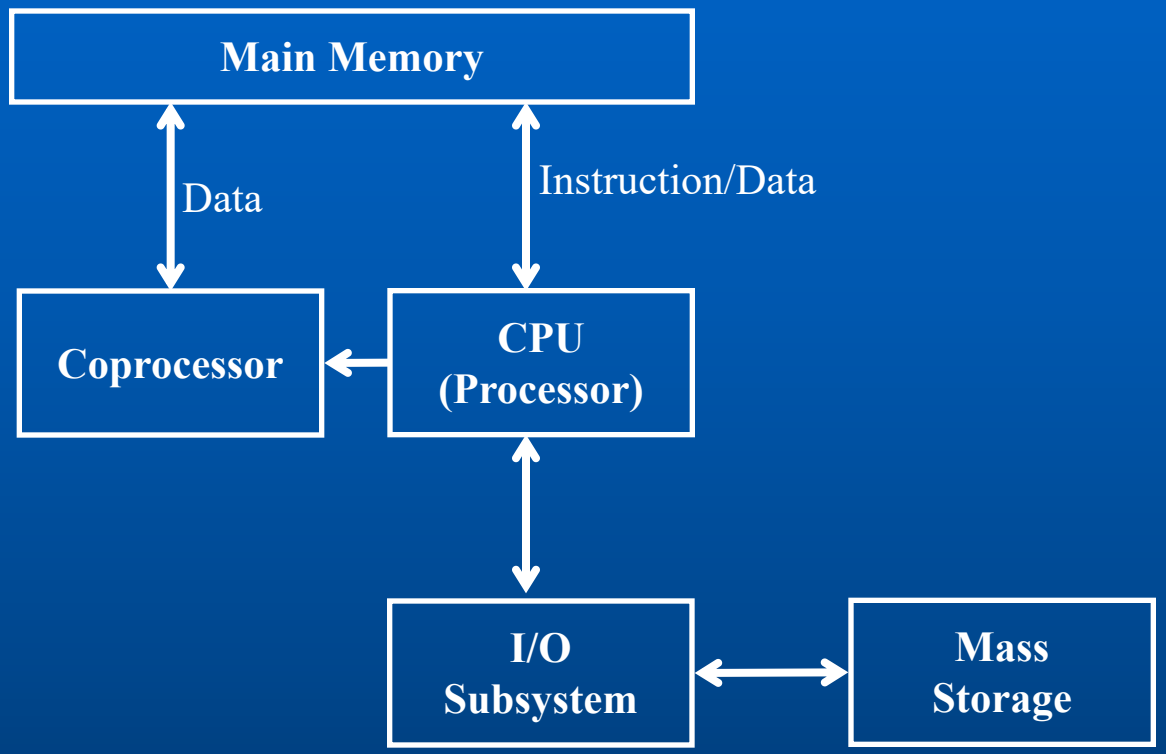
Term	Definition	Unit	Example
Instruction Pipeline Cycle	One processor clock cycle during which all active pipeline stages execute simultaneously	Clock cycle	IF, ID, EX, MEM, WB advance together
Clock Period of the Instruction Pipeline	Time required for one pipeline cycle	Seconds (ns, ps)	2 GHz → 0.5 ns
Instruction Issue Latency	Number of clock cycles between issuing consecutive instructions	Clock cycles	Issue every cycle → 1 cycle latency
Instruction Issue Rate	Number of instructions issued per clock cycle	Instructions/cycle	Dual-issue → 2 instructions/cycle
Simple Operation Latency	Number of clock cycles from the start of an operation until its result is available	Clock cycles	Integer add = 2 cycles, FP multiply = 5 cycles
Resource Conflict	A structural hazard where multiple instructions compete for the same hardware resource simultaneously	—	Two instructions requiring the same multiplier or memory port

Processors and Coprocessors



CPU with built-in floating point unit

Processor and Coprocessor



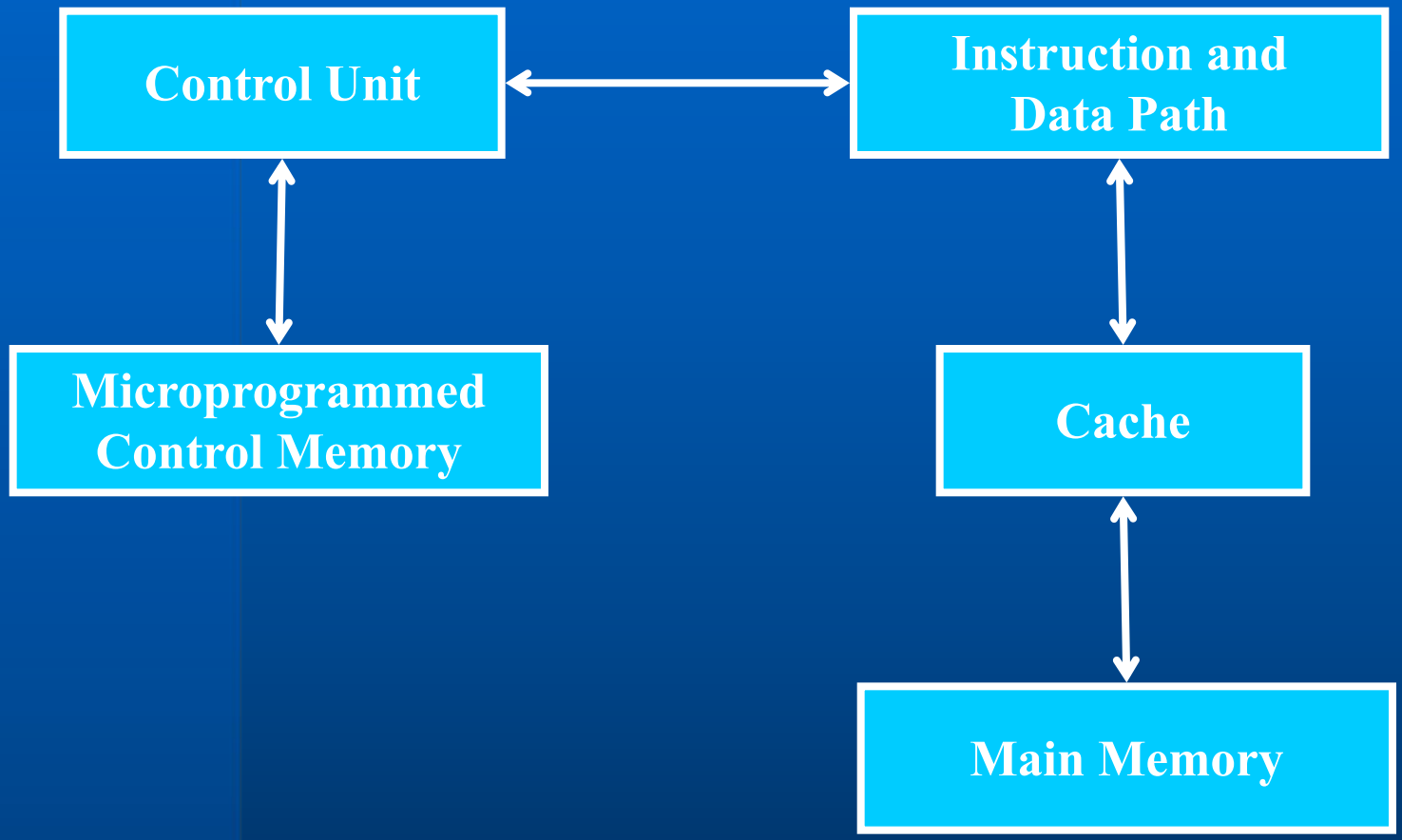
CPU with attached coprocessor

Instruction-Set Architectures

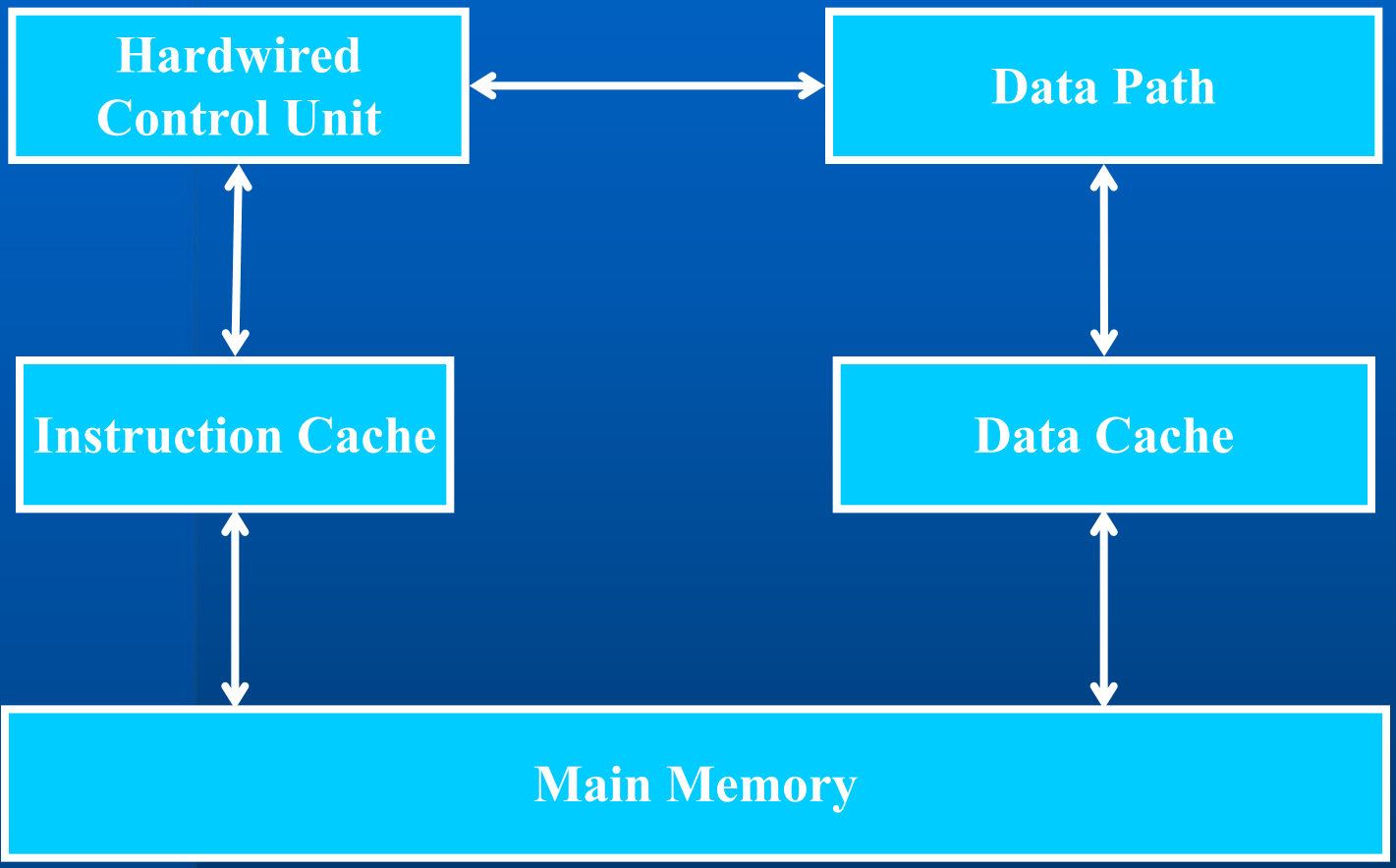
Specifies the primitive commands or machine instructions

- Complex Instruction Sets
  - Most functions have been built into the hardware.
  - Has large instruction set.
- Reduced Instruction Sets
  - Has small instruction set.

Typical CISC Processor Architecture



Typical RISC Processor Architecture

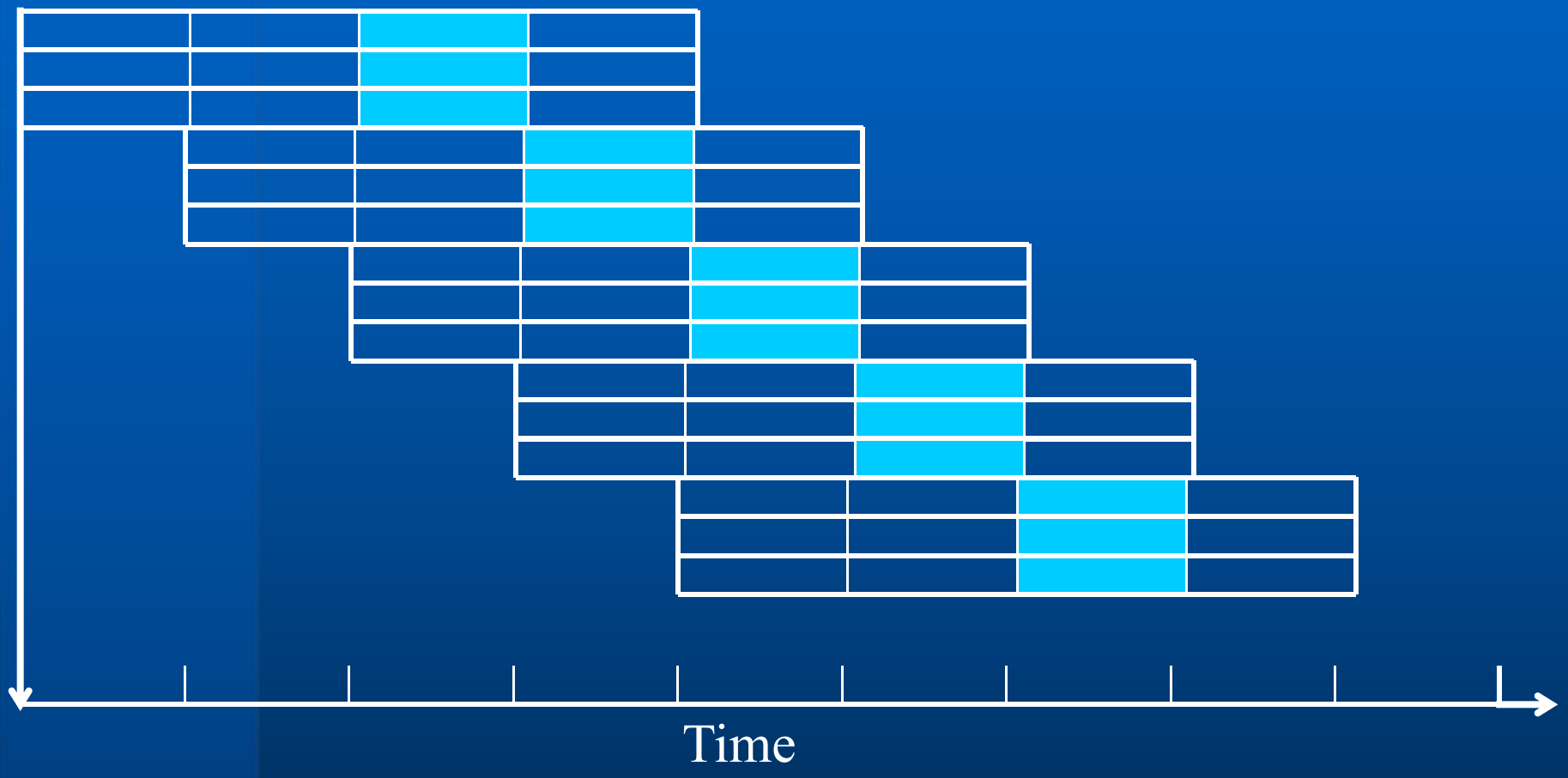


Differences between CISC & RISC architecture

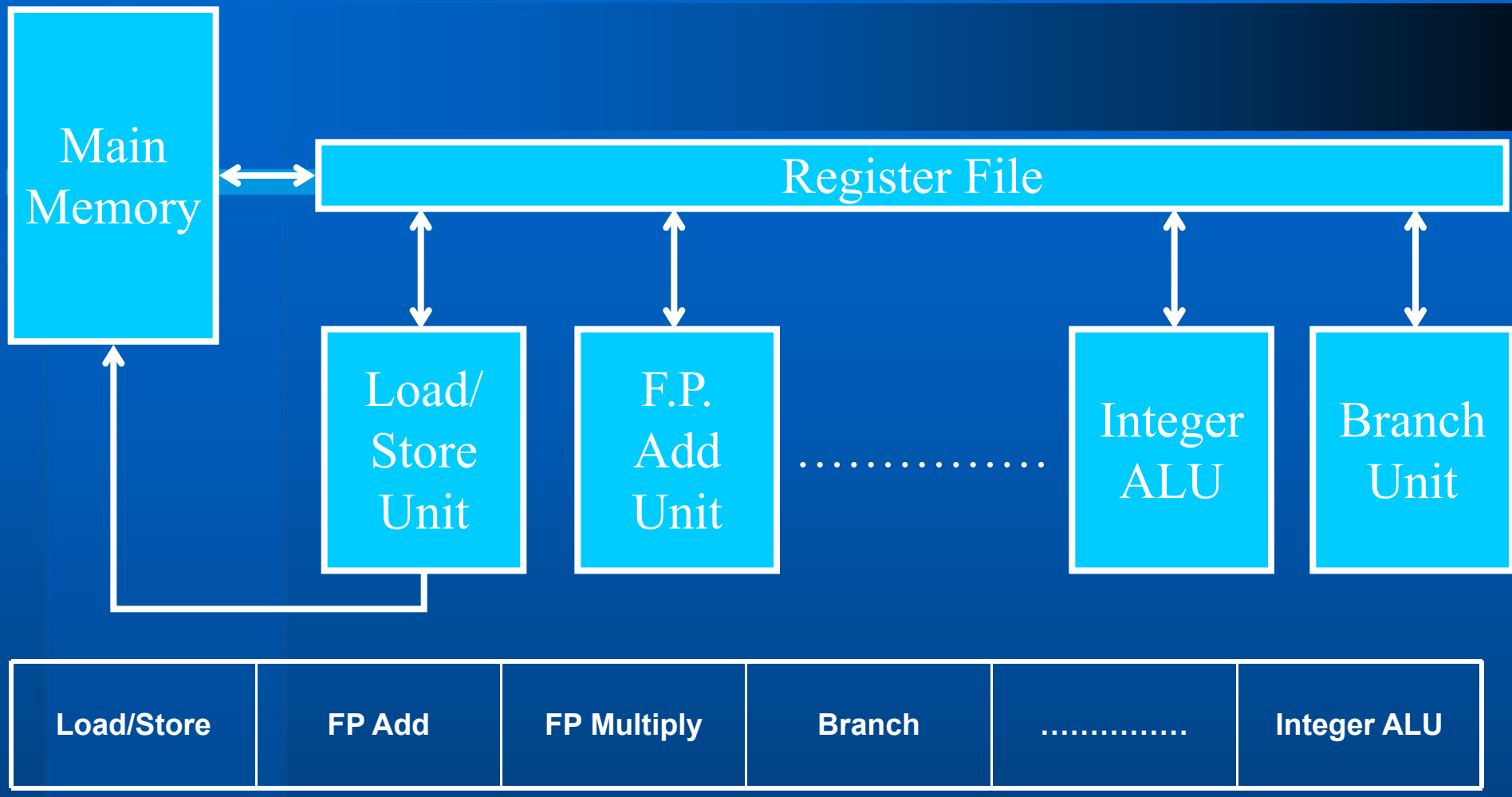
Characteristic	CISC	RISC
Instruction set size & instruction format	Large Set of Instruction with variable formats	Small set of instructions with fixed format
Addressing modes	12-24	Limited to 3-5
General purpose registers and cache design	8-24 GPRs, mostly with a unified cache for instructions and data, recent designs also use split caches	Large numbers (32-192) of GPRs with mostly split data cache and instruction cache
Clock rate and CPI	33-50 MHz in 1992 with a CPI between 2 and 15	50-150 MHz in 1993 with one cycle for almost all instructions and an average CPI < 1.5
CPU Control	Most microcoded using control memory (ROM), but modern CISC also uses hardwired control	Most hardwired without control memory

Superscalar Processor

Designed to exploit more instruction level parallelism in user programs.



VLIW Processor



Typical VLIW processor & instruction format

VLIW Execution

